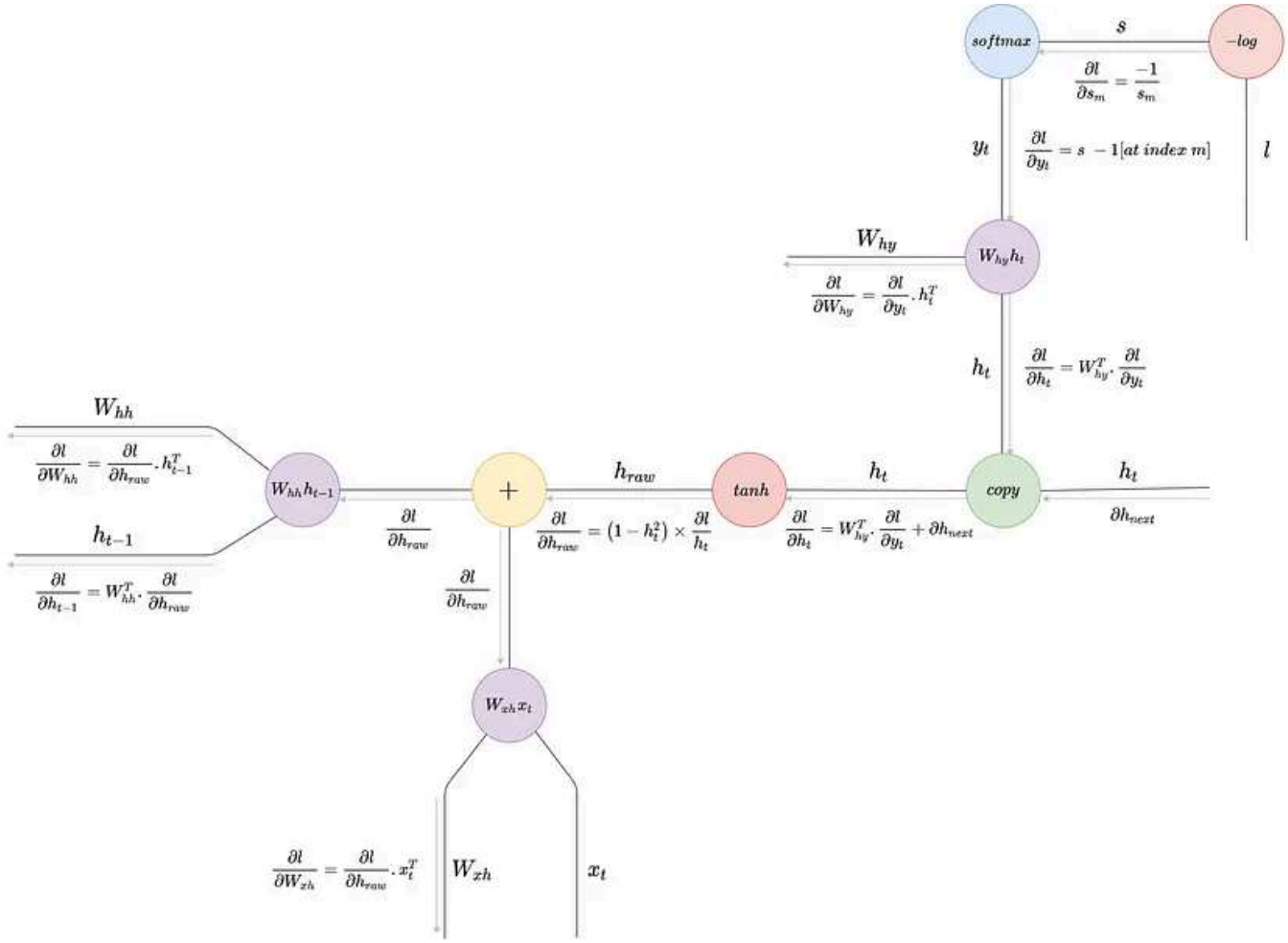
Backpropagation in RNN Explained

A step-by-step explanation of computational graphs and backpropagation in a recurrent neural network

Neeraj Krishna · Follow
Published in Towards Data Science
10 min read · Apr 9, 2022

Listen Share



Backpropagation in RNN

Introduction

In the early days of machine learning when there were no frameworks, most of the time in building a model was spent on coding backpropagation by hand. Today, with

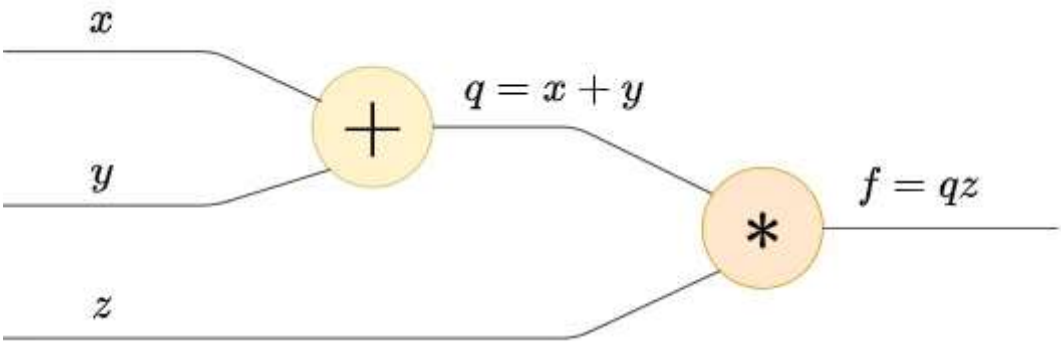
the evolution of frameworks and autograd, to backpropagate through a deep neural network with dozens of layers, we just have to call `loss.backward` — that's it. However, to gain a solid understanding of deep learning, it's crucial we understand

wrt the weights of the network to get the direction of decreasing loss, and during optimization we move along this direction and update the weights thereby minimizing the loss.

In this article we'll understand how backpropation happens in a recurrent neural network.

Computational Graphs

At the heart of backpropagation are operations and functions which can be elegantly represented as a computational graph. Let's see an example: consider the function $f = z(x+y)$; It's computational graph representation is shown below:

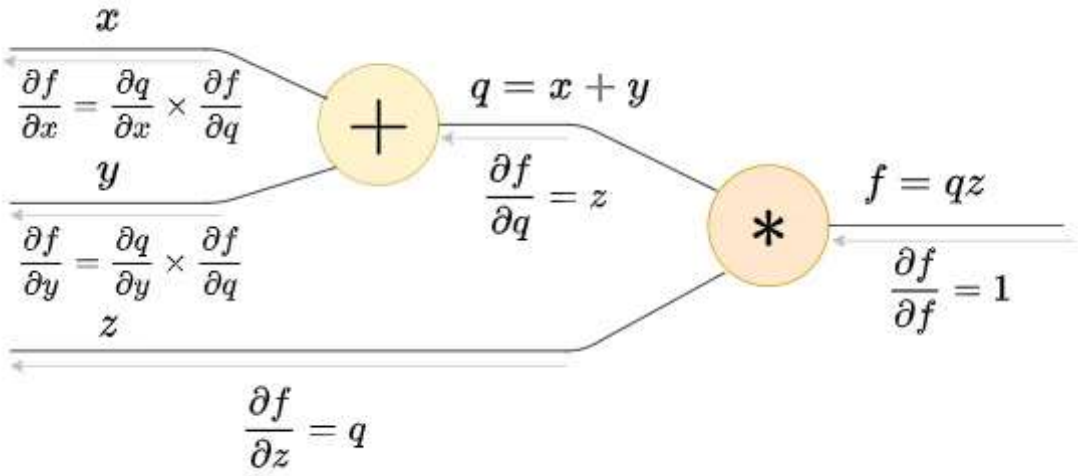


forward pass

A computational graph is essentially a directed graph with functions and operations as nodes. Computing the outputs from the inputs is called the forward pass, and it's customary to show the forward pass above the edges of the graph.

In the backward pass, we compute the gradients of the output wrt the inputs and show them below the edges. Here, we start from the end and go to the beginning computing gradients along the way. Let's do the backward pass for this example.

Notation: let's represent the derivative of a wrt b as $\partial a/\partial b$ throughout the article.

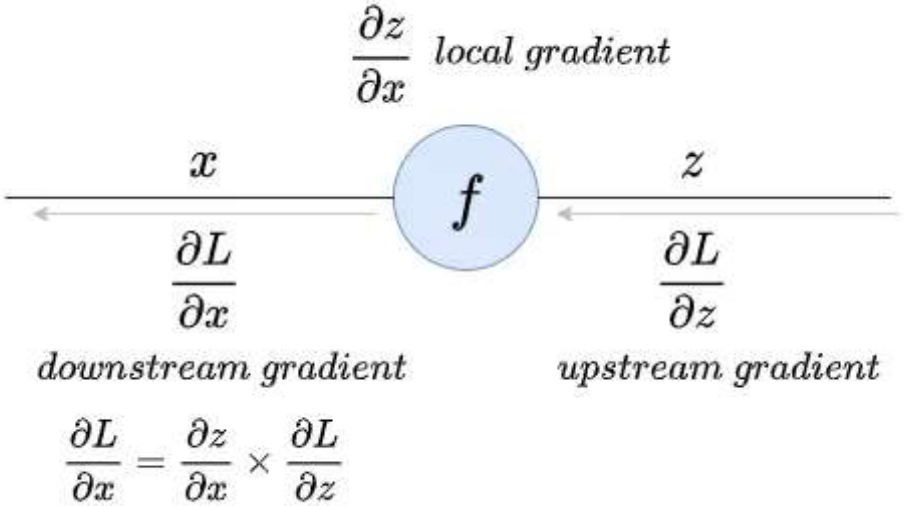


The light gray arrows represent backward pass

First, we start from the end and compute *∂f/∂f* which is 1, then moving backward, we compute *∂f/∂q* which is *z,* then *∂f/∂z* which is *q,* and finally we compute *∂f/∂x and ∂f/∂y.*

Upstream, Downstream, and Local Gradients

If you observe, we cannot compute *∂f/∂x and ∂f/∂y* directly, so we use the chain rule to first compute *∂q/∂x* and then multiply it with *∂f/∂q* which was already computed in the preceding step to get *∂f/∂x.* Here, *∂f/∂q* is called the upstream gradient*, ∂f/∂x* is called the downstream gradient, and *∂q/∂x* is called the local gradient.

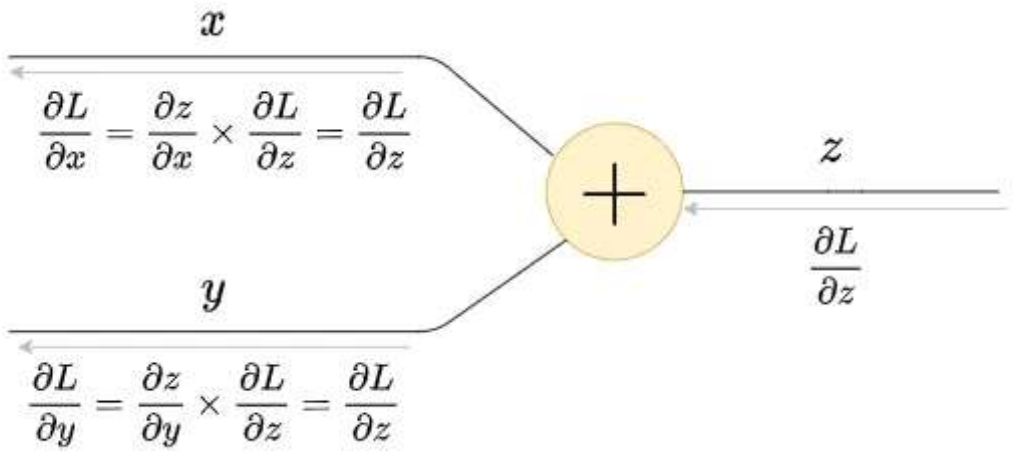



```
downstream gradient = local gradient × upstream gradient
```

Advantages of Computational Graphs

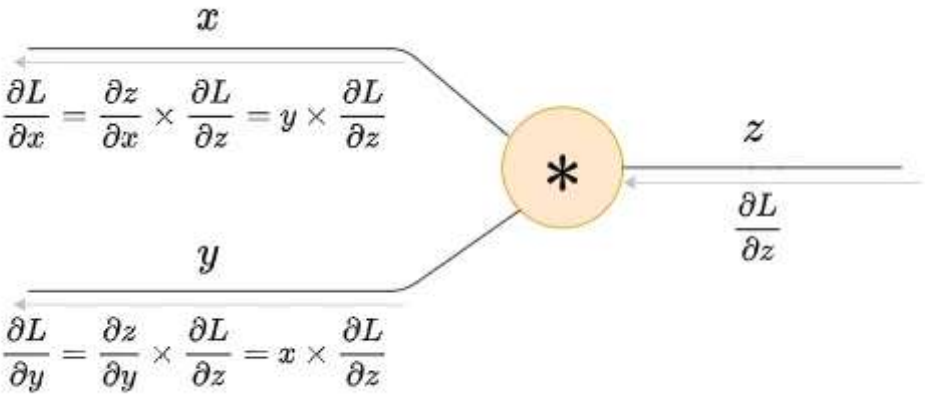
1. **Node-based approach:** In backpropagation, we're always interested in gradient flow, and when working with computational graphs, we can think of gradient flow in terms of nodes rather than functions or operations.

Consider a simple addition node as shown in the below figure:



gradient distributor

Given inputs x and y, the output $z = x + y$. The upstream gradient is $\partial L/\partial z$ where L is the final loss. The local gradient is $\partial z/\partial$x, but since $z = x + y$, $\partial z/\partial x = 1$. Now, the downstream gradient $\partial L/\partial x$ is the product of the upstream gradient and the local gradient, but since the local gradient is unity, the downstream gradient is equal to the upstream gradient. In other words, the gradient flows through the addition node as is, and so the addition node is called the gradient distributor.



gradient swap multiplier

Similarly, for the multiplication node, if you do the calculations, the downstream gradient is the product of the upstream gradient and the other input as shown in the above figure. So, the multiplication node is called the gradient swap multiplier.

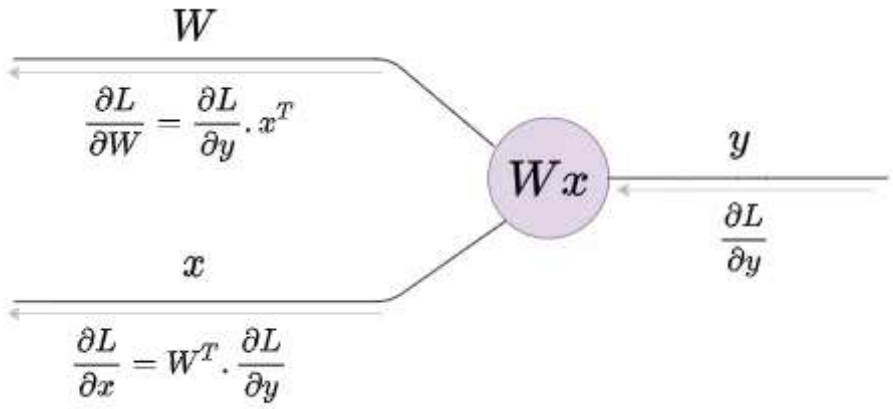
Again, the key takeway here is to think of gradient flow in a computational graph in terms of nodes.

2. **Modular approach:** The downstream gradient at a particular node depends only on its local gradient and the upstream gradient. So if we want to change the architecture of the network in the future, we could simply plug and pull the appropriate nodes without affecting other nodes. This approach is modular, especially when working with autograd.

3. **Custom nodes:** We could combine multiple operations into a single node like a sigmoid node or a softmax node as we'll see next.

Gradients of Vectors and Matrices

When working with neural networks, we usually deal with high dimensional inputs and outputs which are represented as vectors and matrices. The derivative of a scalar wrt a vector is a vector that represents how the scalar is affected by a change in each element of the vector; the derivative of a vector wrt another vector is a Jacobian matrix that represents how each element of the vector is affected by a change in each element of the other vector. Without proof, the gradients are shown below:

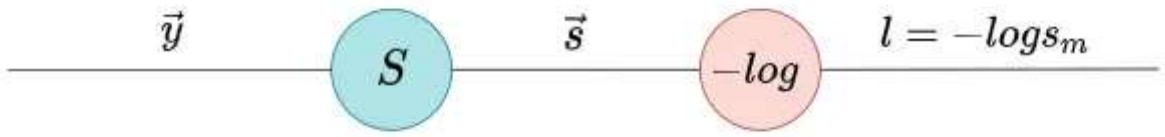


matrix multiplication gradient

Here W is the weight matrix, x is the input vector, and y is the output product vector.

Gradient of Cross-Entropy loss (Optional)

Let's do another example to reinforce our understanding. Let's compute the gradient of a cross-entropy loss node which is a softmax node followed by a log loss node. This is the standard classification head used across many neural networks.



softmax + -ve log loss

Forward pass

In the forward pass, a vector $\vec{y}$ = `[y1, y2, y3, ..., yn]` passes through the softmax node to get the probability distribution `S = [S1, S2, S3, ..., Sn]` as the output. Then, say the ground truth index is m, we take the negative logarithm of `Sm` to compute the loss: `l = -log(Sm)`.

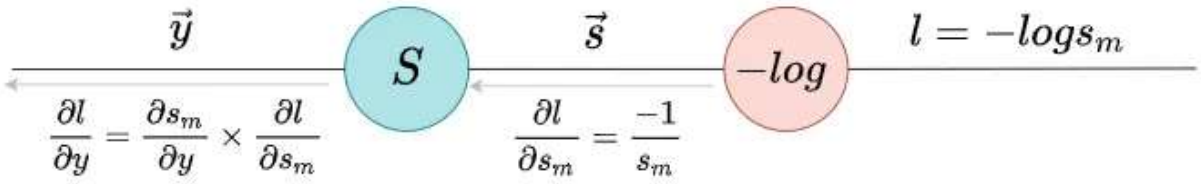
The softmax function is given by:

$$S_m = \frac{e^{y_m}}{\sum_{k=1}^{n} e^{y_k}}$$

Backward pass

The tricky part here is the dependence of loss on a single element of the vector S. So, $l = -log(Sm)$ and $\partial l/\partial Sm = -1/Sm$ where Sm represents the mth element of S where m is the ground truth label. Next, moving back, we ought to compute the gradients

of the softmax node. Here the downstream gradient is $\partial l/\partial y$, the local gradient is $\partial S/\partial y$, and the upstream gradient is $\partial l/\partial Sm$.



backward pass

First, let's compute the local gradient $\partial S/\partial y$. Now, here S is a vector and y is also a vector so $\partial S/\partial y$ will be a matrix that represents how each element of S is affected by a change in each element of y; but you see, the loss depends on only a single element of S at the ground truth index, so we're only interested to find how a single element of S is affected by a change in each element of y. Mathematically, we're interested in finding $\partial Si/\partial y$ where Si is the ith element of the vector S. Here's another catch, Si is simply the softmax function applied over the ith element of $\vec{y}$, which means Si has more dependence on yi and less on the other elements of $\vec{y}$. So we cannot compute $\partial Si/\partial y$ directly. So, we'll find $\partial Si/\partial yj$ where yj is an arbitary element of $\vec{y}$ and consider two cases where $j = i$ and $j \neq i$ as shown below:

Let the vector $\vec{y} = [y_1, y_2, y_3, \ldots, y_n]$

The softmax function on the i_{th} element of the vector, y_i is given by $S_i = \frac{e^{y_i}}{\sum_{k=1}^{n} e^{y_k}}$

Let $h(y) = e^{y_i}$ and $g(y) = \sum_{k=1}^{n} e^{y_k}$

then we can write $S_i = \frac{h(y)}{g(y)}$

$$\implies \frac{\partial S_i}{\partial y_j} = \frac{h'(y)g(y) - g'(y)h(y)}{g(x)^2}$$

Now, $h(y) = e^{y_i}$

$$\implies \frac{\partial h(y)}{\partial y_j} = h'(y) = \left\{ e^{y_i} \text{ if } j = i \text{ else } 0 \right\}$$

also, $g(y) = \sum_{k=1}^{n} e^{y_k}$

$$\implies \frac{\partial g(y)}{\partial y_j} = g'(y) = e^{y_j}$$

case 1: $j = i$

Here, $h'(y) = e^{y_i}$ and $g'(y) = e^{y_j}$

$$\implies \frac{\partial S_i}{\partial y_j} = \frac{e^{y_i} \times \sum_{k=1}^{n} e^{y_k} - e^{y_j} \times e^{y_i}}{\left(\sum_{k=1}^{n} e^{y_k}\right)^2}$$

$$\implies \frac{\partial S_i}{\partial y_j} = \frac{e^{y_i}}{\sum_{k=1}^{n} e^{y_k}} \times \left(1 - \frac{e^{y_j}}{\sum_{k=1}^{n} e^{y_k}}\right)$$

But $S_i = \frac{e^{y_i}}{\sum_{k=1}^{n} e^{y_k}}$ and $S_j = \frac{e^{y_j}}{\sum_{k=1}^{n} e^{y_k}}$

$$\implies \frac{\partial S_i}{\partial y_j} = S_i \left(1 - S_j\right)$$

case 1: j= i

case 2: $j \neq i$

Here, $h'(y) = 0$ and $g'(y) = e^{y_j}$

$$\implies \frac{\partial S_i}{\partial y_j} = \frac{0 \times \sum_{k=1}^{n} e^{y_k} - e^{y_j} \times e^{y_i}}{\left(\sum_{k=1}^{n} e^{y_k}\right)^2}$$

$$\implies \frac{\partial S_i}{\partial y_j} = -\frac{e^{y_j}}{\sum_{k=1}^{n} e^{y_k}} \times \frac{e^{y_i}}{\sum_{k=1}^{n} e^{y_k}}$$

Again $S_i = \frac{e^{y_i}}{\sum_{k=1}^{n} e^{y_k}}$ and $S_j = \frac{e^{y_j}}{\sum_{k=1}^{n} e^{y_k}}$

$$\implies \frac{\partial S_i}{\partial y_j} = -S_j S_i$$

case 2: j ≠ i

So finally we can write:

$$\frac{\partial S_i}{\partial y_j} = \left\{ S_i \left(1 - S_j\right) \text{ if } j = i \text{ else } -S_j S_i \right\}$$

local gradient of the softmax node

Next let's compute the downstream gradient $\partial l/\partial y$. Now, since the downstream gradient is the product of the local gradient and the upstream gradient, let's again find $\partial l/\partial yj$ and consider two cases where $j = i$ and $j \neq i$ as shown below:

The loss function l is given by $l = -\log S_m$
where S_m is the softmax function applied over the m_{th} element of $\vec{y}$

$$S_m = \frac{e^{y_m}}{\sum_{k=1}^{n} e^{y_k}}$$

The upstream gradient is $\frac{\partial l}{\partial S_m} = \frac{-1}{S_m}$

The downstream gradient $\frac{\partial l}{\partial y_j}$ is the product of the local gradient and the upstream gradient.

$$\implies \frac{\partial l}{\partial y_j} = \frac{\partial l}{\partial S_m} \times \frac{\partial S_m}{\partial y_j}$$

case 1: $j = m$
Here the local gradient is $\frac{\partial S_m}{\partial y_j} = S_m\left(1 - S_j\right)$

But since $j = m$, we can write it as $S_m\left(1 - S_m\right)$

$$\implies \frac{\partial l}{\partial y_j} = \frac{-1}{S_m} \times S_m\left(1 - S_m\right)$$

$$\implies \frac{\partial l}{\partial y_j} = S_m - 1$$

case 1: j= i

case 2: $j \neq i$
Here the local gradient is $\frac{\partial S_m}{\partial y_j} = -S_m S_j$

$$\implies \frac{\partial l}{\partial y_j} = \frac{-1}{S_m} \times -S_m S_j$$

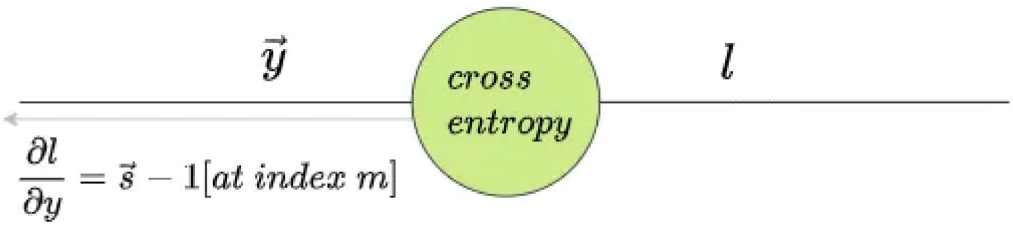
$$\implies \frac{\partial l}{\partial y_j} = S_j$$

case 2: j ≠ i

So finally we can write:

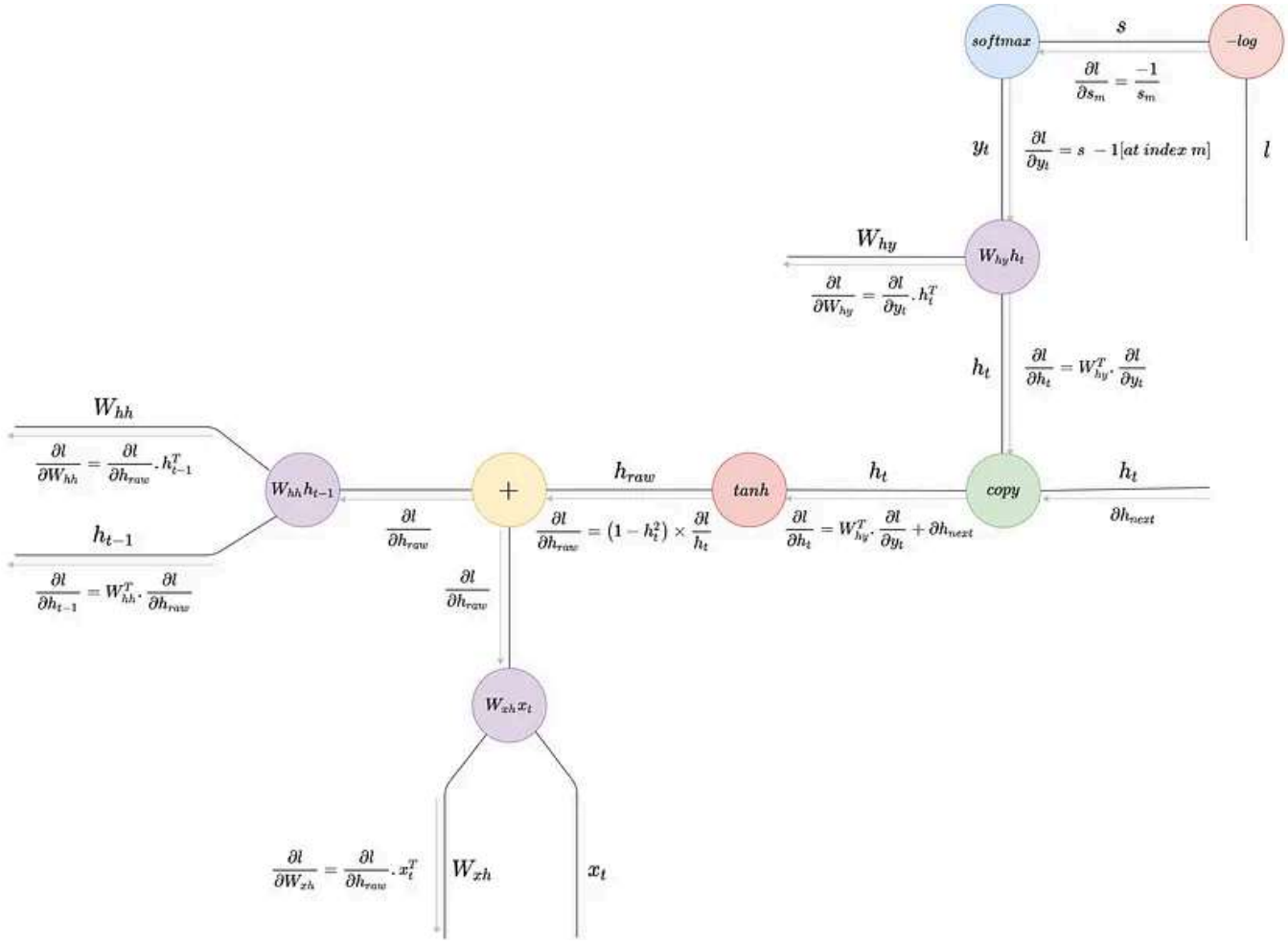
$$\frac{\partial l}{\partial y_j} = \left\{ S_m - 1 \text{ if } j = m \text{ else } S_j \right\}$$

So if the vector $\vec{y}$ = `[y1, y2, y3, ..., ym, ..., yn]` passes through the softmax node to get the probability distribution `S = [S1, S2, S3, ..., Sm, ..., Sn]`, then the downstream gradient *∂l/∂y* is given by `[S1, S2, S3, ..., Sm – 1, ..., Sn]` i.e., we keep all the elements of the softmax vector as is and subtract 1 from the element at the ground truth index. This can also be represented as *S - 1[at index m].* So next time we want to backpropagate through a cross-entropy loss node, we can simply compute the downstream gradient as *S - 1[at index m].*



Alright, now that we've got a solid foundation in backpropagation and computational graphs, let's look at backpropagation in RNN.

Backpropagation in RNN



Forward pass

In the forward pass, at a particular timestep, the input vector and the hidden state vector from the previous timestep are multiplied by their respective weight matrices and are summed up by the addition node. Next, they pass through a non-linear function and then they are copied: one of them goes as an input to the next time step, and the other goes into the classification head where it's multiplied by a weight matrix to obtain the logits vector before computing the cross-entropy loss. This is a typical generative RNN setup where we model the network such that given an input character, it predicts the probability distribution of the next appropriate character. If you're interested to build a character RNN in pytorch, please check my other article here. The forward pass equations are shown below:

$$h_t = tanh(W_{xh}x_t + W_{hh}h_{t-1})$$

$$y_t = W_{hy}h_t$$

$$s = Softmax(y_t)$$

$$l = -logs_m$$

Backward pass

In the backward pass, we start from the end and compute the gradient of the classification loss wrt the logits vector — details of which have been discussed in the previous section. This gradient flows backward to the matrix multiplication node where we compute the gradients wrt both the weight matrix and the hidden state. The gradient wrt the hidden state flows backward to the copy node where it meets the gradient from the previous time step. You see, a RNN essentially processes sequences one step at a time, so during backpropagation the gradients flow backward across time steps. This is called **backpropagation through time**. So, the gradient wrt the hidden state and the gradient from the previous time step meet at the copy node where they are summed up.

Next, they flow backwards to the `tanh` non-linearity node whose gradient can be computed as: $\partial \tanh(x)/\partial x = 1-\tanh^2(x)$. Then this gradient passes through the addition node where it's distributed to both the matrix multiplication nodes of the input vector and the previous hidden state vector. We usually don't compute the gradient wrt the input vector unless there is a special requirement, but we do compute the gradient wrt the previous hidden state vector which then flows back to the previous time step. Please refer to the diagram for the detailed mathematical steps.

Let's see how it looks in code.

Python Code

Andrej Karpathy has implemented character RNN from scratch in Python/Numpy, and his code brilliantly captures the backpropagation step we've discussed as shown below:

Code by Andrej Karpathy. Reused here under BSD license.

The full code can be found here, and the reader is highly encouraged to check it out. If you're looking for a pytorch implementation of RNN with example, please check my other article here.

Why backpropagation in RNN isn't effective

If you observe, to compute the gradient wrt the previous hidden state, which is the downstream gradient, the upstream gradient flows through the tanh non-linearity and gets multiplied by the weight matrix. Now, since this downstream gradient

flows back across time steps, it means the computation happens over and over again at every time step. There are a couple of problems with this:

1. Since we're multiplying over and over again by the weight matrix, the gradient will be scaled up or down depending on the largest singular value of the matrix: if the singular value is greater than 1, we'll face an *exploding gradient* problem, and if it's less than 1, we'll face a *vanishing gradient* problem.
2. Now, the gradient passes through the tanh non-linearity which has saturating regions at the extremes. It means the gradient will essentially become zero if it has a high or low value once it passes through the non-linearity — so the gradient cannot propagate effectively across long sequences and it leads to ineffective optimization.

There is a way to avoid the exploding gradient problem by essentially "clipping" the gradient if it crosses a certain threshold. However, RNN still cannot be used effectively for long sequences.

I hope you've got a clear idea of how backpropagation happens in RNN. Let me know if you've any doubts. Let's connect on Twitter and LinkedIn.

Image Credits

All the images used in this article are made by the author.

Science Mathematics Deep Learning Machine Learning Data Science

789K Followers · Last published 2 hours ago

Your home for data science and AI. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

Follow

Written by Neeraj Krishna

890 Followers · 92 Following

I write about effective learning, technology, and deep learning | 2x top writer | senior data scientist @MakeMyTrip

Responses (7)

What are your thoughts?

Respond

Harsh Lunia
over 2 years ago (edited)

The upstream gradient is ∂L/∂x where L is the final loss. The local gradient is ∂z/∂x, but since z = x + y, ∂z/∂x = 1

Here Upstream gradient should be ∂L/∂z

7 1 reply Reply

D. Sigdel
over 2 years ago

Impressive article on computational graph on RNN.

1 Reply

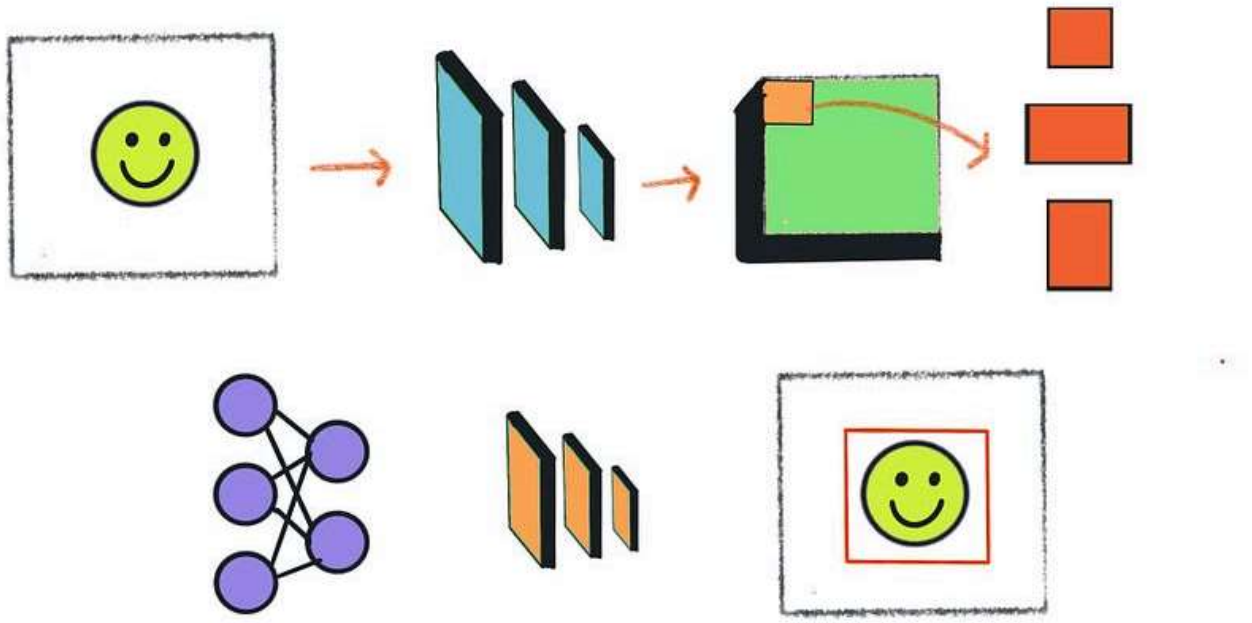
B Badrinath Goteti
7 months ago

to much complicated explanation

Reply

See all responses

More from Neeraj Krishna and Towards Data Science



In Towards Data Science by Neeraj Krishna

Understanding and Implementing Faster R-CNN: A Step-By-Step Guide

Demystifying Object Detection

Nov 2, 2022 665 13

In Towards Data Science by Amanda Iglesias Moreno

How to Run Jupyter Notebooks and Generate HTML Reports with Python Scripts

A step-by-step guide to automating Jupyter Notebook execution and report generation using Python

2d ago 173 2

In Towards Data Science by James Barney

Are Meta's AI Profiles Unethical?

As AI becomes further enmeshed into every product we use, what rules should exist to protect humans?

2d ago 181 4

In New Writers Welcome by Neeraj Krishna

Please Don't Take Notes While Listening to Lectures. Do this Instead.

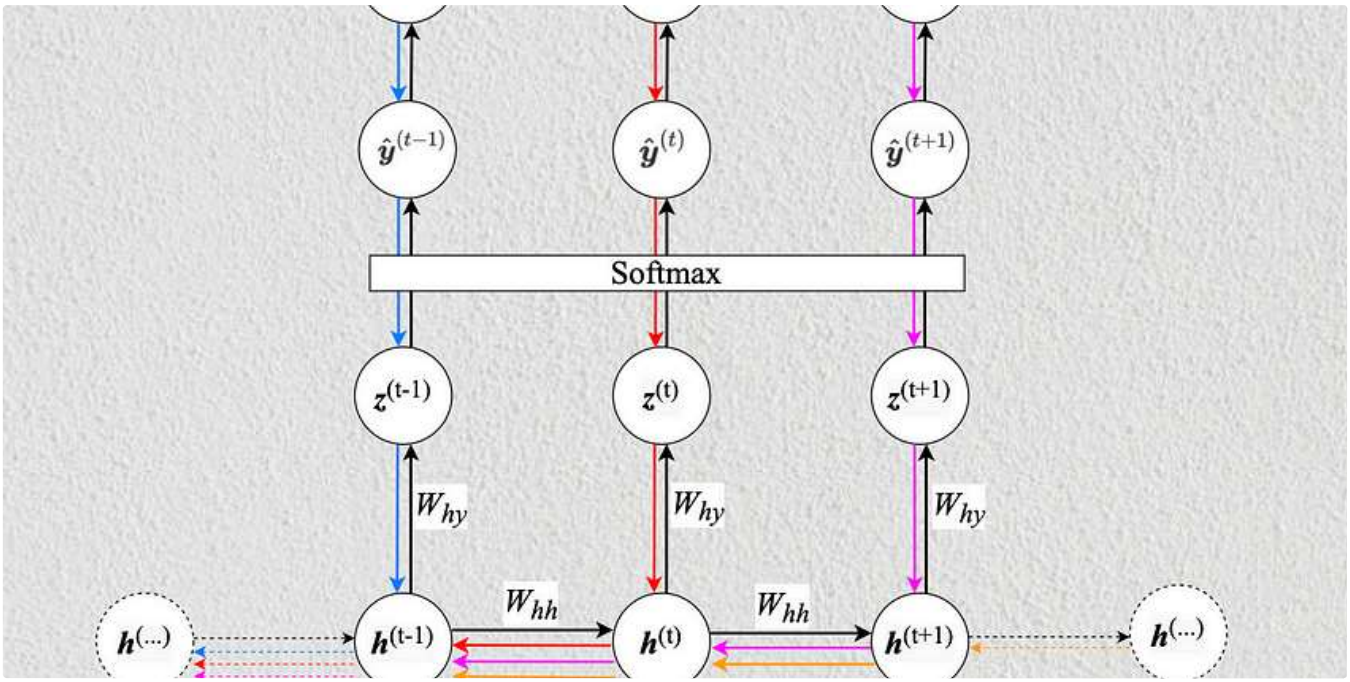
The right way to take notes

Dec 5, 2022 553 12

See all from Neeraj Krishna

See all from Towards Data Science

Recommended from Medium

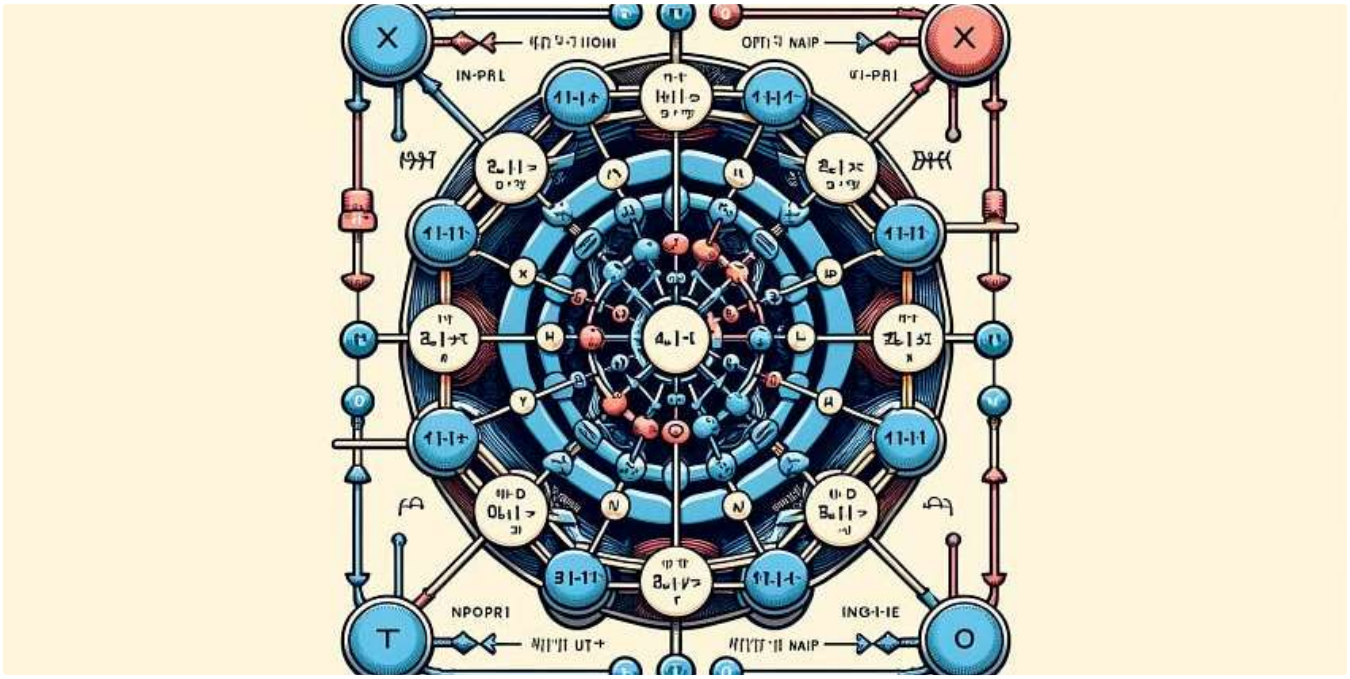


Liyuan Chen

Backpropagation Through Time in RNN

Mathematical notation and programming syntax often represent differently for the same concepts. In this blog, I'll explain BPTT in RNN...

Oct 28, 2024 2



In Towards Data Science by Cristian Leo

The Math Behind Recurrent Neural Networks

Dive into RNNs, the backbone of time series, understand their mathematics, implement them from scratch, and explore their applications

Apr 27, 2024 816 3

Lists

Predictive Modeling w/ Python

20 stories · 1769 saves

Practical Guides to Machine Learning

10 stories · 2140 saves

Natural Language Processing

1884 stories · 1534 saves

data science and AI

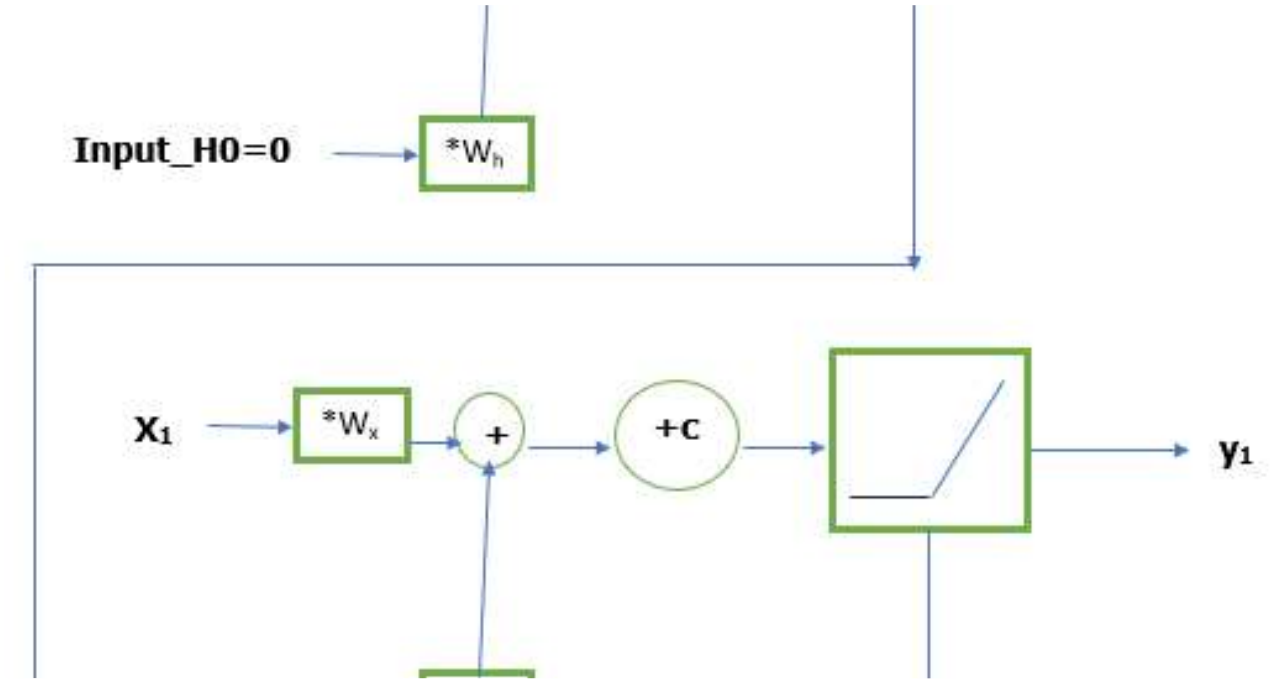
40 stories · 313 saves

Mikel

Pinterest ML Internship Summer 2025

Looking for a tech internship for the Summer 2025. I share my recent Interview experience, Questions, Solutions and tips.

Oct 27, 2024 7 2

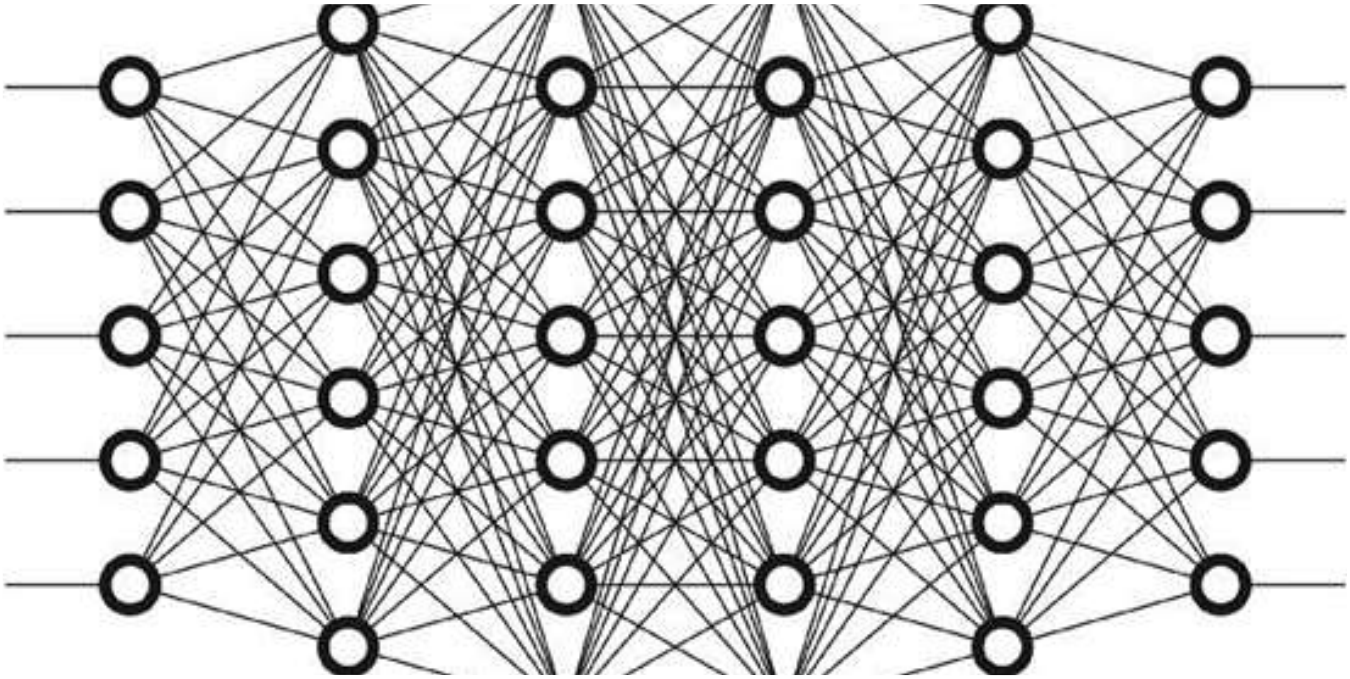


Rohollah

Natural Language Processing Series Part 3 : A Beginner's Guide to Understanding RNNs

A Beginner's Guide to Understanding RNNs and Their Problems, Like Exploding and Vanishing Gradients

Sep 28, 2024 51



In Data And Beyond by Chinmay Bhalerao

Say "HELLO" to neurons | Deep Learning : 0

This blog explores the fundamentals of neurons, exploring their structure and essential terminologies. Let's enter the fascinating world...

Oct 10, 2024 253 1

In Programming Domain by Hivan du

27. Deep Learning Advanced—Why RNN

This article is section 27 of “Hivan’s AI Compendium—AI Core Foundations”

Sep 2, 2024 12

See more recommendations